Hard questions — honest answers

Each answer says what is verified and where. If the answer is "we do not know" or "it does not work yet", it is said that way.

Why not Pharo? Pharo is the live Smalltalk environment and protoST does not compete with it. protoST explores something else: running Smalltalk code from files, on a core (protoCore) with native threads with no global lock and other languages alongside. If what you need is the live environment, use Pharo.

Where is the image? And the IDE? There is no image, by design: the program is the file. There is a REPL (`protost -i`), a command-line debugger (`protost -d`) and a DAP adapter for VS Code (`protost --dap`, `docs/debugging.md`). There is no class browser.

Performance compared with Pharo? We did not measure against Pharo and we will not give a figure we did not measure. We measured against CPython 3.14 with the same algorithm, the same N and the same verified result (`benchmarks/reports/2026-10-12-protocore-2.14.6.md`: synthetic benchmarks, on a laptop with a 6-core AMD Ryzen 5 5500U, with the published protoST binary on protoCore 2.14.6 and on 2.14.5):

- startup: 25 ms (CPython: 32–35 ms);
- whole-process time: from 0.9× to 21–22× that of CPython, geometric mean 3.2–3.3× in two runs; with small N, startup weighs heavily in each time;
- subtracting each runtime's startup, the work itself takes about 4–7× (integer loop), 30–35× (`fib`, recursive method dispatch) and 55–64× (exceptions) as long as in CPython; the geometric mean is between 14× and 20× depending on the run (with small N the subtraction is sensitive to noise);
- in parallel, twelve actors run 2.8× faster with the pool than with one worker, and `saturation_big` reaches 2.5× with four workers and 2.9× with six, no ceiling up to six.

Since 0.6.2 protoST uses protoCore's adaptive heap controller and sizes each thread's frame array on demand (K6): against 0.6.1, actor programs run 16–43 % faster with a fifth to a half of the memory (`benchmarks/reports/2026-10-12-adaptive-heap-default.md`). Looking for a small actor regression turned up a real bug, fixed in 0.6.1: a recursion that never loops (the textbook fib) never had its garbage reclaimed; fib 29 ran out of memory (`benchmarks/reports/2026-10-11-actor-regression-bisection.md`).

In other words: on one thread protoST is considerably slower than CPython. What we show is that the same code uses several cores.

Why does demo 2 show less speedup than before? Because the single-thread code got faster. protoCore 2.14.6 stopped creating temporaries when it compares, divides or takes the modulo of small integers, and demo 2 is exactly that (`isPrime` in a loop). Measured with the same binary on both protoCore versions, 10 alternating rounds on an idle machine in AC mode (12/10; medians):

	protoCore 2.14.5	protoCore 2.14.6	change
one thread	1358 ms	716 ms	47 % faster
twelve actors	476 ms	293 ms	38 % faster
speedup shown	2.8×	2.4×	

(The earlier 4.4× and 3.8× figures were measured on a busy machine: load slows the one-thread half, which cannot spread out, and inflates the ratio.)

Both times improve; the one-thread time improves more, so the ratio drops. Parallelism did not get worse: the twelve actors finish sooner than ever. On the rest of the benchmarks 2.14.6 changes nothing beyond the noise except exceptions (7 % faster) and `parallel_speedup` with the pool (8 %). The protoCore report measured the same in another session (one thread 33 % and twelve actors 26 % faster, ratio 2.99× → 2.66×); the talk's figures are the idle ones above.

Before 2.14.6 those temporaries made the demo depend on glibc: in the pre-talk audit the one-thread half ran 10 % faster with Ubuntu 24.04's glibc 2.39 than with the laptop's 2.43 (`benchmarks/reports/2026-10-12-pre-talk-063.md`); with 2.14.6 they no longer allocate. And with the machine busy every configuration dropped to 2–3×: the figure on screen depends on the machine and its load.

What is the GC like? protoCore's: concurrent, with its own thread. Since 0.6.2 protoST lets protoCore's adaptive controller size the heap: a budget of 75 % of physical memory (or of the process limit) and, inside it, a soft limit that starts at 128 MB and grows only while the program waits for the collector. If the live data does not fit in the budget, it exits with a clear message. But the peak is no longer capped at 640 MB: a loop that keeps allocating reached 0.88–3.5 GB in three runs (report of 12/10). If a fixed size matters, `PROTOCORE_HEAP_LIMIT_CELLS=<cells>` sets a ceiling (64-byte cells: 10000000 is 640 MB of cells), and `PROTOCORE_ADAPTIVE_HEAP=0` restores 0.6.1's 640 MB ceiling (`KNOWN_ISSUES.md`, K5). Note that this ceiling counts cells, not the process's memory. With `PROTOCORE_ADAPTIVE_HEAP=0`, `(1 to: 500000) asArray` runs out of memory; with protoCore 2.14.5 its peak RSS reached 1,165 MB (the audit's 1.19 GB: 1,192,800 kB), because the collector's mark work list was a vector that doubled outside the cell heap (up to 512 MB). protoCore 2.14.6 bounds it to a fixed 8 MiB stack: the same program peaks at 690 MB and exits in 3.0 s instead of 5.9 s (report of 12/10 on 2.14.6, five runs). What remains above the cells is now the list of marked cells and the rest of the process. It was not used up to 0.6.1 because under it `fib.st` took three times as long; the cause was protoST's (a recursion did not hand over its garbage), fixed in 0.6.1. The sweep, the most expensive phase, walks several chains at once and can add helper threads (2.14). The stop-the-world pauses we measured in protoST on the benchmarks stay below 0.27 ms; before 0.6.2 they reached 20–31 ms with six to twelve workers, because each pause scanned a whole 1 M-slot frame array per thread (`KNOWN_ISSUES.md`, K6, resolved).

Why prototypes? protoCore is a prototype model; protoST classes are named prototype objects. To the Smalltalk programmer they look like classes (`subclass:`, class methods, class variables and class-instance variables). The differences this produces are in chapter 14.

What does the protoCore ecosystem compare with? What are its references? Its references are conceptual, not performance ones, because protoCore does not optimise the current way of working: it tries to change the foundation. It is a bridge between functional programming and mutable languages.

- **Rich Hickey and Clojure** are the closest reference. Their model of identity, state and value says that an identity points to a succession of immutable values. Clojure offers it *inside* a functional language, with explicit `atom`s and `ref`s. protoCore puts it *beneath* mutable languages: a mutable object in Python, JavaScript or Smalltalk is, by construction, an atomic reference to an immutable snapshot, and the programmer does not change how they write. The code stays imperative; the semantics underneath are those of values.
- **Persistent data structures** (Okasaki; Bagwell's trees) make it cheap for a "modification" to create a new version that shares almost everything with the old one.
- **Self and Lieberman's prototypes** give an object model with no imposed classes, which lets very different languages share objects.
- **Erlang/BEAM, as a contrast,** reaches safe concurrency the opposite way: it shares nothing and copies messages. protoCore shares everything without copying, because what is shared is immutable.

The result is ordinary imperative code with safe concurrency, native threads without a global lock, and objects that cross from one language to another in the same process, by construction rather than by discipline.

What about GraalVM? And the JVM? They are not comparable in what matters. GraalVM is a general-purpose platform optimised for existing languages and built entirely on the JVM's object model. It does not propose another way of working: it optimises the current one to the extreme, and does it very well. protoCore changes the foundation. If the question is speed, they win without discussion: protoCore has no JIT and does not compete on single-thread performance (see the performance question).

Who uses it? Nobody in production. It is an experiment and a protoCore demonstrator; we are looking for people to try it with real programs.

What is missing? `thisContext`, `outer` (today it behaves like `pass`), the full metaclass hierarchy, `become:`, image and browser (by design), `Process`, `Semaphore`, `Delay` and `fork` (concurrency is actors), classes such as `IdentityDictionary`, `ByteArray` or `ScaledDecimal`, and installed runtimes loading each other (the core allows it since protoCore 2.6: embedded in one process, protoScala reads and writes protoST objects without copying them; `docs/INTEROP.md` §0). What we found is in chapter 14 of the tutorial and in `docs/STATUS.md`; it is not a guarantee that there are no other differences.

Can it talk to the outside world? Since 0.5.0, yes: files and directories (`'data.csv' asFileReference`), standard input and output (`Stdio stdin nextLine`: a script is a Unix filter), arguments and environment variables, exit codes, other programs (`OSProcess`), TCP and UDP sockets, TLS, and HTTP client (https too) and server, with

one actor per connection. An I/O wait inside an actor does not hold up the GC, and the worker pool grows while it lasts, so many actors waiting for replies do not starve the ones that produce them. HTTP/2, WebSockets and a TLS server are missing. Demo 4 shows it. The operating-system layer is a shared library, protoIO, which protoScala and protoClojure use too: the same input and output exists, under each language's own names, in the three runtimes.

Does it run on Windows? The published package (0.6.5) is for Linux; on Windows it runs under WSL2 with Ubuntu 24.04, installing the same `.deb` packages. The sources build natively with Visual Studio 2022, and continuous integration runs the suite on Windows: 1097 cases, all but the ones that depend on POSIX tools (Ctrl-C sent with `kill`, an example in `sh` syntax) and the terminal console test; `docs/INSTALLATION.md`, "Windows (MSVC)". There is no published Windows package yet.

Is it stable? What open bugs does it have? 0.5.0 passed its 1068 `ctest` cases; 0.6.5 (with protoCore 2.14.6) passes its 1101 cases on Linux: conformance programs, unit tests, the examples, command-line tests and the documentation examples. Continuous integration runs 1100 on Linux and on macOS (plus a parallelism proof measured with a clock, which does not gate on a shared runner) and 1097 on Windows. Those are tests, not the audit's programs: while preparing this talk, an adversarial audit of about 1,100 programs, in two rounds, found silent wrong results and hangs; every fix for a wrong result, a crash or a hang has its regression test. The intermittent hang S19 was caught with the process still alive and turned out to be a lost wakeup in libstdc++ 13's `std::counting_semaphore`; it was replaced. Its test was kept out of the CI gate while evidence accumulated: it passed on the first attempt in the 59 CI runs since the fix and 150 times in a row locally, and since 0.6.3 it gates CI again. And 0.6.3 found, thanks to a compiler warning, a latent defect: the computed-goto dispatch did not release the collector's transient pins, and in the GCC builds on Linux those pins stopped protecting objects once the kernel had loaded; it is fixed, with its test. What is open is in `docs/STATUS.md`.

Can actors block each other? An actor that does `wait` inside a method does not handle other messages until it receives the answer. If two actors wait on each other nobody makes progress: protoST detects it and signals an `Error` ("deadlock") instead of hanging. Inside actors it is better to chain with `thenDo:` or `whenAll:`.

License? MIT.

How do I contribute? Write programs the way you would write them and report what surprises you (issues in the repository). `docs/ROADMAP.md` lists the open work.